

Praxis: Power Management im Treiber

Runtime-PM-Pfad eines realen Treibers read-only auditieren

5.07

Der Audit beantwortet vier konkrete Fragen

01 Welches Gerät und welcher Treiber?

sysfs-Pfad, Busbindung und Quellpfad eindeutig zuordnen.

02 Wann wechselt der Runtime-Zustand?

Status und Zeitwerte vor, während und nach einem Zugriff beobachten.

03 Sind Referenzen und Callbacks korrekt?

Datenpfad, Ressourcenreihenfolge und Fehlerpfade im Quelltext prüfen.

04 Welche Evidenz trägt die Bewertung?

Befehle, Fundstellen und beobachtete Übergänge gemeinsam dokumentieren.

Abgabe: ein ausgefülltes Auditblatt mit Befund, Beleg und einem priorisierten Risiko.

Drei Leitplanken halten die Übung sicher

SYSTEM

Laborgerät wählen

Nur ein entbehrliches Gerät verwenden.
Produktive Datenträger, Netzwerkpfade
und Eingabegeräte ausschließen.

ZUGRIFF

Read-only bleiben

power/control,
Autosuspend-Verzögerung und
Wakeup-Konfiguration nicht verändern.

ABBRUCH

Nebenwirkungen stoppen

Bei I/O-Fehlern, Hängern oder
unerwartetem Wakeup sofort beenden
und Zustand sichern.

Diese Einheit bewertet den PM-Pfad. Sie erzwingt weder Runtime-Suspend noch System-Sleep.

Gerät und Treiber eindeutig zuordnen

TERMINAL

```
$ dev=/sys/bus/.../devices/...  
$ test -d "$dev/power" && echo "PM-Daten vorhanden"  
$ readlink -f "$dev/driver"  
$ readlink -f "$dev/driver/module" 2>/dev/null  
$ udevadm info -q property -p "$dev" | head
```

01 Pfad

Der sysfs-Gerätepfad bleibt die gemeinsame Referenz für alle Beobachtungen.

02 Bindung

driver und module verbinden das Gerät mit dem später geprüften Quelltext.

Ohne eindeutige Bindung lassen sich sysfs-Befund und Quellcode nicht belastbar zusammenführen.

Die Baseline zeigt Policy, Zustand und bisherige Laufzeit

READ-ONLY BASELINE

```
$ for f in runtime_status control autosuspend_delay_ms \  
>         runtime_active_time runtime_suspended_time wakeup; do  
>   test -r "$dev/power/$f" || continue  
>   printf '%-25s ' "$f"  
>   cat "$dev/power/$f"  
> done
```

01 Policy

control=on verhindert
Autosuspend. control=auto
erlaubt ihn nur grundsätzlich.

02 Evidenz

Zeitwerte liefern eine
Baseline. Ein einzelner
runtime_status ist nur eine
Momentaufnahme.

Fehlende power-Dateien sind ein Befund: Nicht jedes Gerät exportiert dieselben PM-Daten.

Runtime-Zustände beobachten

BEOBACHTUNG

```
$ while sleep 1; do
>   printf '%s  ' "$(date +%T)"
>   cat "$dev/power/runtime_status"
> done

# In einem zweiten Terminal:
# Einen normalen, sicheren Gerätezugriff auslösen.
```

01 Leerlauf

Nach der Verzögerung kann active in suspended wechseln. Das Gerät muss dies unterstützen.

02 Zugriff

Ein zulässiger Zugriff sollte das Gerät aktivieren und danach wieder freigeben.

03 Abweichung

Bleibt es aktiv, kommen Policy, Referenzen, Arbeit oder Abhängigkeiten als Ursache infrage.

Der gebundene Treiber führt zum relevanten Quellpfad

TREIBERQUELLE

```
$ drv=$(basename "$(readlink -f "$dev/driver")")
$ mod=$(basename "$(readlink -f "$dev/driver/module")")
$ printf 'driver=%s module=%s\n' "$drv" "$mod"
$ modinfo -n "$mod"

$ cd "$ksrc"
$ rg -n "$drv|$mod" drivers/
```

01 Kernelstand

Quelltext und laufender Kernel müssen zusammenpassen. Distribution-Patches beachten.

02 Fundstelle

Bus-Treiber, Datenstruktur und dev_pm_ops gemeinsam erfassen.

Eine ähnliche Upstream-Datei ersetzt nicht den Quellstand des tatsächlich laufenden Kernels.

Der I/O-Pfad braucht einen Referenzvertrag

QUELLTEXTSUCHE

```
$ rg -n "pm_runtime_(resume_and_get|get_sync)" "$src"  
$ rg -n "pm_runtime_put(_autosuspend|_sync|_noidle)?" "$src"  
$ rg -n "goto|return|err_|out_" "$src"
```

01 Get

Erfolg bedeutet: Das Gerät ist aktiv und der Usage Counter schützt den Zugriff.

02 I/O-Region

Jeder sleep-fähige Bus- oder Registerzugriff liegt innerhalb der Referenz.

03 Put

Auch Fehler- und Abbruchpfade geben genau die erworbene Referenz zurück.

Markiere jeden Get-Pfad und folge ihm bis zum passenden Put – nicht nur im Erfolgsfall.

Ressourcenfolge der PM-Callbacks

SUSPEND

RESUME

01	Neue Transfers blockieren	I/O freigeben	05
02	IRQ-Quelle deaktivieren	Register synchronisieren	04
03	Registerzugriff sperren	Reset lösen	03
04	Reset setzen	Clock aktivieren	02
05	Clock / Versorgung aus	Versorgung einschalten	01

Nur Ressourcen abschalten, die der Callback selbst aktiviert oder eindeutig übernommen hat.

System-Sleep und Runtime-PM verbinden

QUELLTEXTSUCHE

```
$ rg -n "dev_pm_ops|DEFINE_.*PM_OPS" "$src"
$ rg -n "pm_runtime_force_(suspend|resume)" "$src"
$ rg -n "SYSTEM_SLEEP_PM_OPS|suspend|resume" "$src"
```

01 Zuordnung

Prüfen, ob der Bus-Treiber die gefundene dev_pm_ops-Struktur tatsächlich verwendet.

02 Wiederverwendung

Force-Callbacks sind plausibel, wenn Runtime-Suspend denselben Hardwarezustand erzeugt.

03 Sonderfall

Wakeup-fähige Geräte können IRQ- und Zustandslogik während System-Sleep abweichend behandeln.

Makros und Callback-Empfehlungen immer gegen die verwendete Kernelversion prüfen.

Wakeup hält nur den freigegebenen Ereignispfad aktiv

WAKEUP AUDIT

```
$ cat "$dev/power/wakeup" 2>/dev/null  
$ rg -n "device_(init|may)_wakeup" "$src"  
$ rg -n "enable_irq_wake|disable_irq_wake" "$src"  
$ rg -n "pm_wakeup_event|wakeup_source" "$src"
```

01 Freigabe

device_may_wakeup trennt erlaubtes Wakeup von einem normalen Interrupt.

02 Symmetrie

enable_irq_wake und disable_irq_wake brauchen passende Zustände und Fehlerpfade.

03 Accounting

Wakeup Events müssen zum realen Ereignispfad passen, nicht zu jedem IRQ.

Nebenläufigkeit im PM-Pfad

NEBENLÄUFIGKEIT

```
$ rg -n "request.*irq|threaded_irq" "$src"  
$ rg -n "work_struct|schedule_work|queue_work" "$src"  
$ rg -n "cancel_.*work_sync|flush_work" "$src"  
$ rg -n "mutex_(lock|unlock)|spin_lock" "$src"  
$ rg -n "remove|pm_runtime_disable" "$src"
```

01 Schützen

Lock und PM-Referenz müssen gemeinsam genutzte Daten und Hardwarezugriffe abdecken.

02 Stoppen

Suspend und Remove beenden oder synchronisieren IRQ, Timer und Workqueues.

03 Reihenfolge

Keine sleep-fähige PM-Anforderung unter einem dafür ungeeigneten Lock auslösen.

Das Auditblatt verbindet Befund, Beleg und Bewertung

Prüffeld	Erwartung	Beleg	Bewertung
Runtime-Zustand	active und suspended beobachtbar	sysfs-Werte mit Zeitstempel	offen
PM-Referenzen	Get und Put auf allen Pfaden balanciert	Datei und Zeilennummern	offen
Callbacks	Ressourcenfolge plausibel gespiegelt	Suspend-/Resume-Fundstellen	offen
Wakeup / Remove	Ereignis- und Abbaupfad vollständig	API-Fundstellen und Risiko	offen

Eine Bewertung ohne reproduzierbaren Beleg bleibt eine Vermutung.

Vier Fragen schließen den Audit ab

01 Referenz

Welcher Get-/Put-Vertrag schützt den tatsächlichen I/O-Pfad?

02 Reihenfolge

Welche Ressource bestimmt Suspend und Resume, und warum?

03 Nebenläufigkeit

Welche Arbeit kann mit Suspend oder Remove konkurrieren?

04 Risiko

Welcher einzelne Befund verdient als Erstes eine genauere Prüfung?

Ein sicherer Energieübergang ist durch Referenz, Reihenfolge und beobachtbare Evidenz begründet.